MCA Semester III – Data Science Practical Assignment

Exploratory Data Analysis, Visualisation and Data Preprocessing Using Python

Student Name: Yatharth
Enrollment No.: 25030501038
Batch / Division: 2

Dataset: Titanic Passenger Dataset
Domain: Passenger survival / classification
Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

Task 1 – Dataset Introduction

The **Titanic Passenger Dataset** contains information about 891 passengers aboard the RMS Titanic. It is widely used for introductory data-science and machine-learning exercises.

- **Source/reference:** DataScienceDojo Titanic CSV (public GitHub mirror of the well-known Kaggle Titanic training data)
- **Source URL:** `https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv`
- **Records:** 891
- **Attributes in the raw dataset:** 12
- **Domain:** Transportation / passenger survival analysis
- **Purpose:** Explore passenger characteristics and identify patterns related to survival.
- **Target variable:** `Survived` (`0 = No`, `1 = Yes`)

Important attributes

Attribute	Description
PassengerId	Unique passenger identifier
Survived	Survival status; target variable
Pclass	Ticket class (1 = First, 2 = Second, 3 = Third)
Name	Passenger name
Sex	Passenger sex
Age	Age in years
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Ticket	Ticket number
Fare	Passenger fare
Cabin	Cabin number
Embarked	Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

Task 2 – Import Required Libraries

- **Pandas**: data loading, cleaning, transformation and analysis.
- **NumPy**: numerical operations and array utilities.
- **Matplotlib**: base plotting library.
- **Seaborn**: statistical visualisation built on Matplotlib.
- **Scikit-learn preprocessing**: encoding and feature scaling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler

%matplotlib inline
```

Task 3 – Read the Dataset

```
df = pd.read_csv("titanic.csv")
print("Dataset successfully loaded.")
```

```
Dataset successfully loaded.
```

Task 4 – Display First Five Records

`head()` quickly shows the first rows, helping verify that the file was loaded correctly and that columns appear as expected.

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

Next steps: Generate code with df | New interactive sheet

Task 5 – Display Last Five Records

`tail()` checks the final records and is useful for spotting footer rows, truncation issues, or unusual values near the end of a dataset.

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C

Task 6 – Display Random Records

Random sampling gives a less biased quick look than only viewing the beginning or end of the file.

```
df.sample(5, random_state=42)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
709	710	1	3	Moubarek, Master. Halim Gonios ("William George")	male	NaN	1	1	2661	15.2458	NaN	C
439	440	0	2	Kvillner, Mr. Johan Henrik Johannesson	male	31.0	0	0	C.A. 18723	10.5000	NaN	S
840	841	0	3	Alhomaki, Mr. Ilmari Rudolf	male	20.0	0	0	SOTON/O2 3101287	7.9250	NaN	S
				Harper, Miss. Annie								

Task 7 – Number of Rows and Columns

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Number of rows: 891
Number of columns: 12
```

Task 8 – Column Names and Brief Description

The dataset contains identifiers, numerical variables, categorical variables, and one binary target.

```
print(df.columns.tolist())
```

```
attribute_description = pd.DataFrame({
    "Column": df.columns,
    "Description": [
        "Unique passenger identifier",
        "Target: 0 = died, 1 = survived",
        "Passenger ticket class: 1, 2, or 3",
        "Passenger full name",
        "Passenger sex",
        "Passenger age in years",
        "Number of siblings/spouses aboard",
        "Number of parents/children aboard",
        "Ticket number",
        "Passenger fare",
        "Cabin number",
        "Embarkation port: C, Q, or S"
    ]
})
attribute_description
```

```
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
```

	Column	Description
0	PassengerId	Unique passenger identifier
1	Survived	Target: 0 = died, 1 = survived
2	Pclass	Passenger ticket class: 1, 2, or 3
3	Name	Passenger full name
4	Sex	Passenger sex
5	Age	Passenger age in years
6	SibSp	Number of siblings/spouses aboard
7	Parch	Number of parents/children aboard
8	Ticket	Ticket number
9	Fare	Passenger fare
10	Cabin	Cabin number
11	Embarked	Embarkation port: C, Q, or S

Next steps: Generate code with `attribute_description` New interactive sheet

Task 9 – Dataset Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Task 10 – Identify Data Types

```
print(df.dtypes)

column_classification = pd.DataFrame({
    "Column": df.columns,
    "Role": [
        "Identifier", "Target", "Categorical (ordinal)", "Identifier/Text",
        "Categorical", "Numerical", "Numerical", "Numerical",
        "Identifier/Text", "Numerical", "Categorical/Text", "Categorical"
```

```
    ]
})
column_classification
```

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

	Column	Role
0	PassengerId	Identifier
1	Survived	Target
2	Pclass	Categorical (ordinal)
3	Name	Identifier/Text
4	Sex	Categorical
5	Age	Numerical
6	SibSp	Numerical
7	Parch	Numerical
8	Ticket	Identifier/Text
9	Fare	Numerical
10	Cabin	Categorical/Text
11	Embarked	Categorical

Next steps: Generate code with `column_classification` New interactive sheet

Task 11 – Descriptive Statistics

For numerical variables, `describe()` reports count, mean, standard deviation, minimum, quartiles, median (50%), and maximum. For object/categorical variables, it reports count, number of unique values, most frequent value, and its frequency.

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
df.describe(include='object')
```

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Dooley, Mr. Patrick	male	347082	G6	S
freq	1	577	7	4	644

Task 12 – Unique Values and Frequency

```
for col in ["Sex", "Embarked", "Pclass"]:
    print(f"\n--- {col} ---")
    print("Unique values:", df[col].unique())
    print("Number of unique values:", df[col].nunique(dropna=True))
    print(df[col].value_counts(dropna=False))
```

```
--- Sex ---
Unique values: ['male' 'female']
Number of unique values: 2
Sex
male      577
female    314
Name: count, dtype: int64

--- Embarked ---
Unique values: ['S' 'C' 'Q' nan]
Number of unique values: 3
Embarked
S      644
C      168
Q       77
NaN      2
Name: count, dtype: int64

--- Pclass ---
Unique values: [3 1 2]
Number of unique values: 3
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

Task 13 – Missing Value Analysis

```
missing_table = pd.DataFrame({
    "Column": df.columns,
    "Missing Values": df.isnull().sum().values,
    "Missing Percentage": (df.isnull().sum().values / len(df)) * 100
}).sort_values("Missing Values", ascending=False)

missing_table
```

	Column	Missing Values	Missing Percentage
10	Cabin	687	77.104377
5	Age	177	19.865320
11	Embarked	2	0.224467
0	PassengerId	0	0.000000
3	Name	0	0.000000
2	Pclass	0	0.000000
1	Survived	0	0.000000
4	Sex	0	0.000000
7	Parch	0	0.000000
6	SibSp	0	0.000000
9	Fare	0	0.000000
8	Ticket	0	0.000000

Next steps: Generate code with `missing_table` New interactive sheet

```
plt.figure(figsize=(10, 5))
plt.imshow(df.isnull().T, aspect="auto", interpolation="nearest")
plt.title("Missing Value Visualization")
plt.xlabel("Passenger Rows")
plt.ylabel("Columns")
plt.show()
```



Task 14 – Missing Value Handling

```
df_clean = df.copy()

df_clean["Deck"] = df_clean["Cabin"].str[0].fillna("Unknown")

age_median = df_clean["Age"].median()
df_clean["Age"] = df_clean["Age"].fillna(age_median)

embarked_mode = df_clean["Embarked"].mode()[0]
df_clean["Embarked"] = df_clean["Embarked"].fillna(embarked_mode)

df_clean["Cabin"] = df_clean["Cabin"].fillna("Unknown")

print("Age median used:", age_median)
print("Embarked mode used:", embarked_mode)
print("\nMissing values after handling:")
print(df_clean.isnull().sum())
```

```
Age median used: 28.0
Embarked mode used: S

Missing values after handling:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
Deck           0
dtype: int64
```

Task 15 – Duplicate Records

```
duplicate_before = df_clean.duplicated().sum()
print("Duplicate records:", duplicate_before)
display(df_clean[df_clean.duplicated()].head())

df_clean.drop_duplicates(inplace=True)

duplicate_after = df_clean.duplicated().sum()
print("Duplicate records after cleaning:", duplicate_after)
```

```
Duplicate records: 0
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Deck

```
Duplicate records after cleaning: 0
```

Task 16 – Univariate Analysis

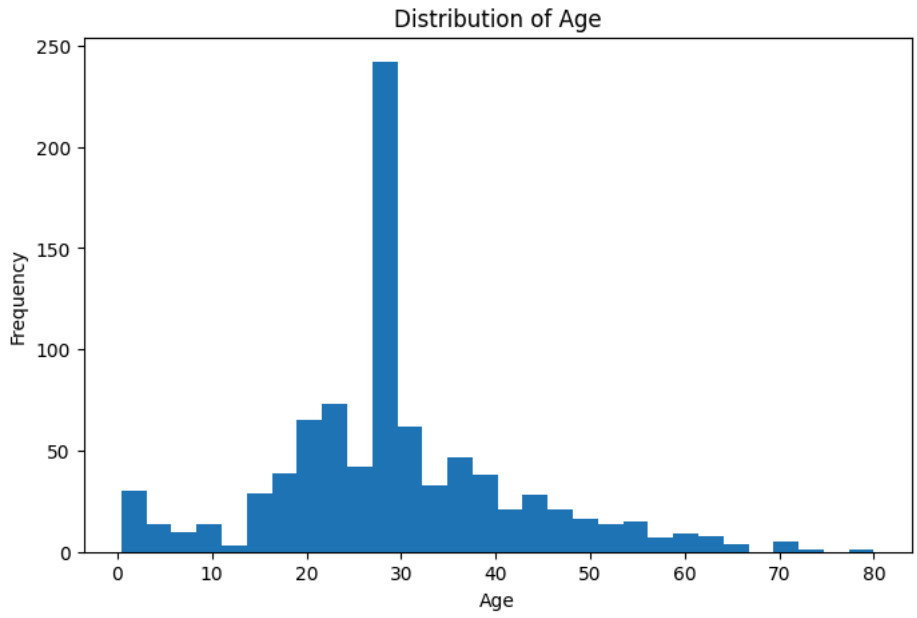
```
univariate_summary = pd.DataFrame({
    "Variable": ["Age", "Fare", "Sex", "Pclass"],
    "Type": ["Numerical", "Numerical", "Categorical", "Categorical"],
    "Key Summary": [
        f"Median age = {df_clean['Age'].median():.1f}; mean age = {df_clean['Age'].mean():.2f}",
        f"Median fare = {df_clean['Fare'].median():.2f}; max fare = {df_clean['Fare'].max():.2f}",
        f"Most common sex = {df_clean['Sex'].mode()[0]}",
        f"Most common class = {int(df_clean['Pclass'].mode()[0])}"
    ]
})
univariate_summary
```

	Variable	Type	Key Summary
0	Age	Numerical	Median age = 28.0; mean age = 29.36
1	Fare	Numerical	Median fare = 14.45; max fare = 512.33
2	Sex	Categorical	Most common sex = male
3	Pclass	Categorical	Most common class = 3

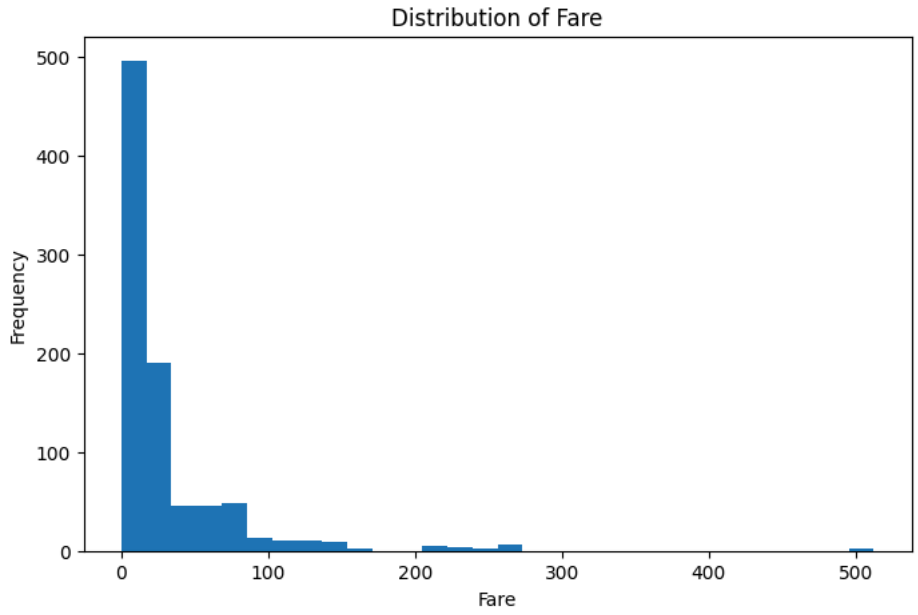
Next steps: Generate code with `univariate_summary` | New interactive sheet

Task 17 – Numerical Data Visualization

```
plt.figure(figsize=(8, 5))
plt.hist(df_clean["Age"], bins=30)
plt.title("Distribution of Age")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



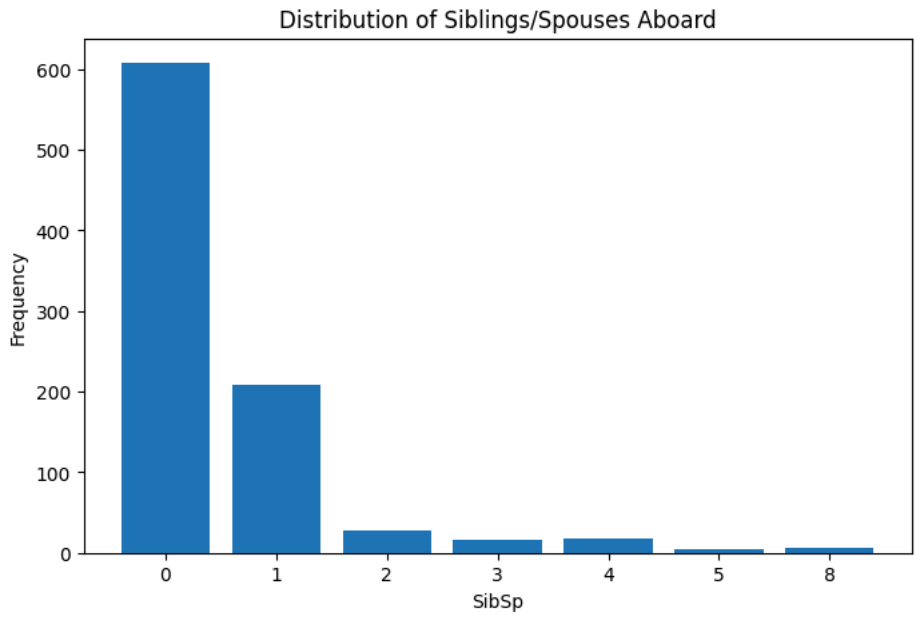

```
plt.figure(figsize=(8, 5))
plt.hist(df_clean["Fare"], bins=30)
plt.title("Distribution of Fare")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()
```



Fare observations:

1. Fare is strongly right-skewed.
2. Most passengers paid relatively low fares.
3. A small number of very high fares create a long upper tail.

```
plt.figure(figsize=(8, 5))
values = df_clean["SibSp"].value_counts().sort_index()
plt.bar(values.index.astype(str), values.values)
plt.title("Distribution of Siblings/Spouses Aboard")
plt.xlabel("SibSp")
plt.ylabel("Frequency")
plt.show()
```



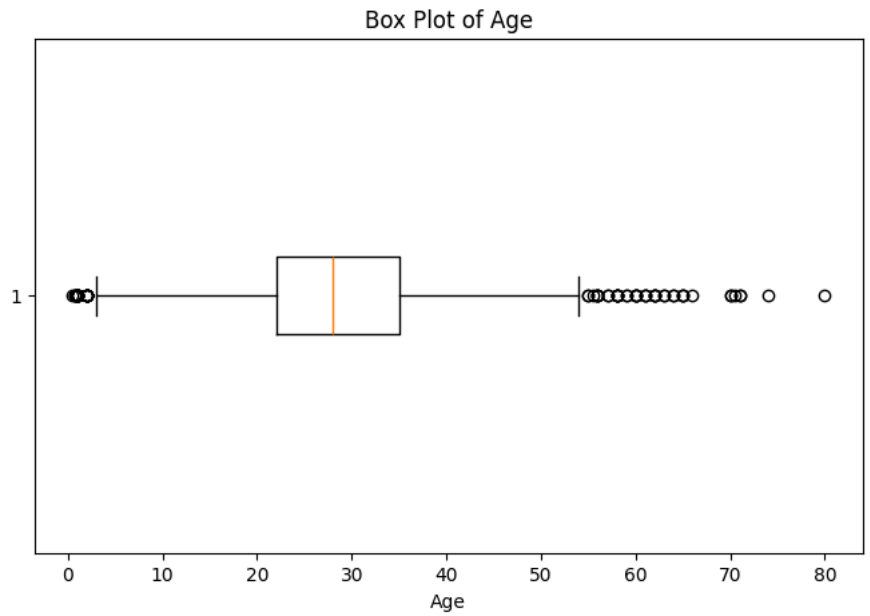
SibSp observations:

1. Most passengers travelled with no sibling or spouse.
2. Values of 1 are also common.
3. Large family-group values are rare.

Task 18 – Box Plot and Outlier Analysis

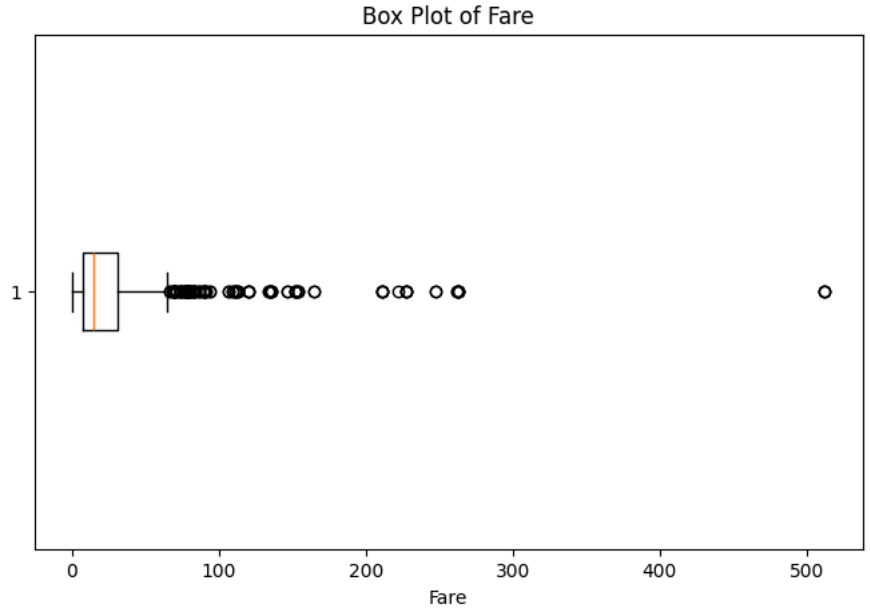
```
plt.figure(figsize=(8, 5))
plt.boxplot(df_clean["Age"], vert=False)
plt.title("Box Plot of Age")
```

```
plt.xlabel("Age")
plt.show()
```



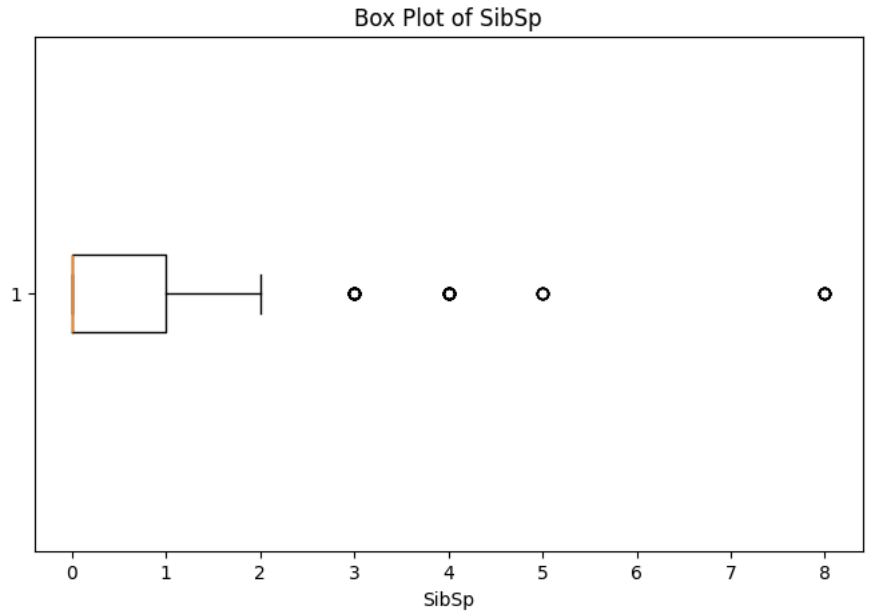
Age: median is around 28 years; the central spread is concentrated in adulthood, with a small number of high-age observations beyond the upper whisker.

```
plt.figure(figsize=(8, 5))
plt.boxplot(df_clean["Fare"], vert=False)
plt.title("Box Plot of Fare")
plt.xlabel("Fare")
plt.show()
```



Fare: the box is compressed toward lower fares and many high-fare observations appear beyond the upper whisker, confirming strong right skew.

```
plt.figure(figsize=(8, 5))
plt.boxplot(df_clean["SibSp"], vert=False)
plt.title("Box Plot of SibSp")
plt.xlabel("SibSp")
plt.show()
```



SibSp: the median is 0 and most values are 0–1; larger family counts are uncommon and appear as statistical outliers.

Task 19 – IQR-Based Outlier Detection

```
def iqr_outlier_summary(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
    return pd.Series({
        "Q1": Q1,
        "Q3": Q3,
        "IQR": IQR,
        "Lower Bound": lower_bound,
        "Upper Bound": upper_bound,
        "Outlier Count": len(outliers)
    })

pd.DataFrame({
    "Age": iqr_outlier_summary(df_clean, "Age"),
    "Fare": iqr_outlier_summary(df_clean, "Fare")
}).T
```

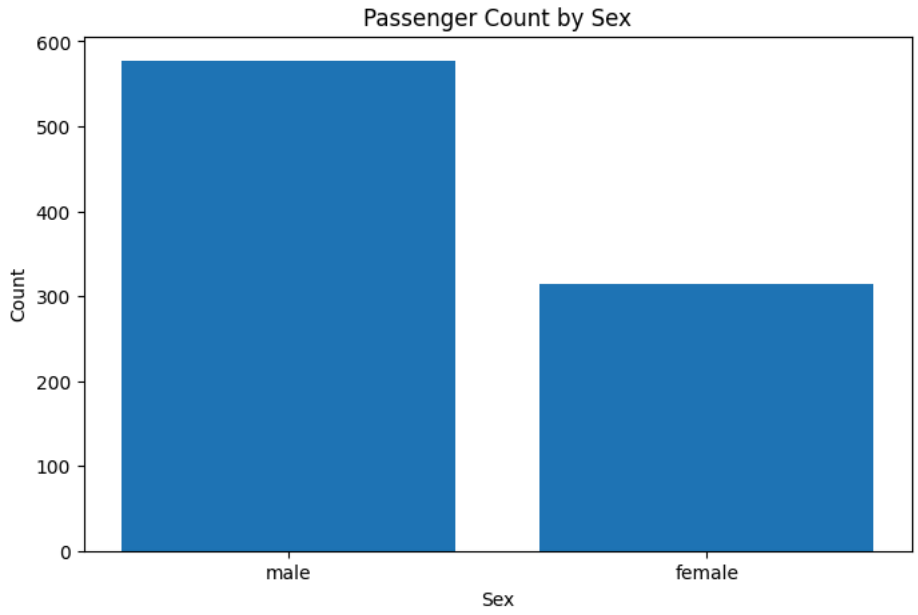
	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count
Age	22.0000	35.0	13.0000	2.500	54.5000	66.0
Fare	7.9104	31.0	23.0896	-26.724	65.6344	116.0

Outlier decision

- **Age:** high-age observations are plausible real passengers, not obvious data-entry errors. They are retained.
- **Fare:** expensive first-class/group tickets can legitimately have very high fares. These values are retained rather than removed or capped.
- This follows the assignment requirement to avoid removing outliers merely to make the dataset look cleaner.

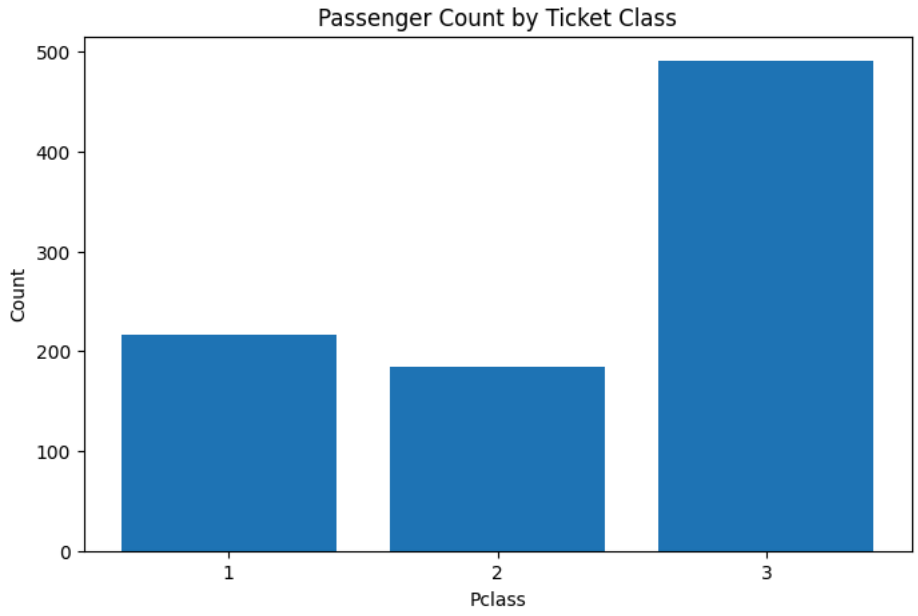
Task 20 – Categorical Data Visualization

```
plt.figure(figsize=(8, 5))
counts = df_clean["Sex"].value_counts()
plt.bar(counts.index.astype(str), counts.values)
plt.title("Passenger Count by Sex")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()
```



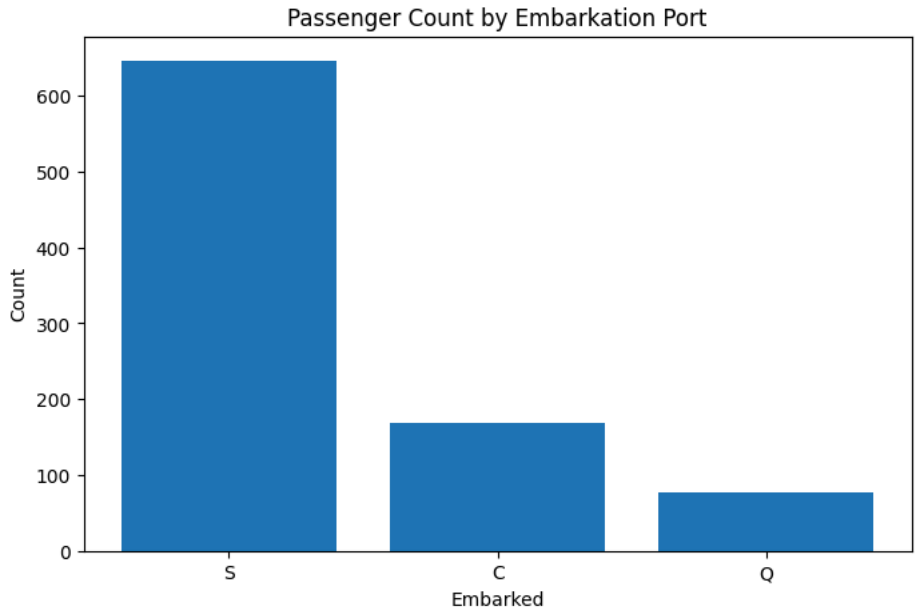
Observation: male passengers are more numerous than female passengers.

```
plt.figure(figsize=(8, 5))
counts = df_clean["Pclass"].value_counts().sort_index()
plt.bar(counts.index.astype(str), counts.values)
plt.title("Passenger Count by Ticket Class")
plt.xlabel("Pclass")
plt.ylabel("Count")
plt.show()
```



Observation: third class contains the largest number of passengers.

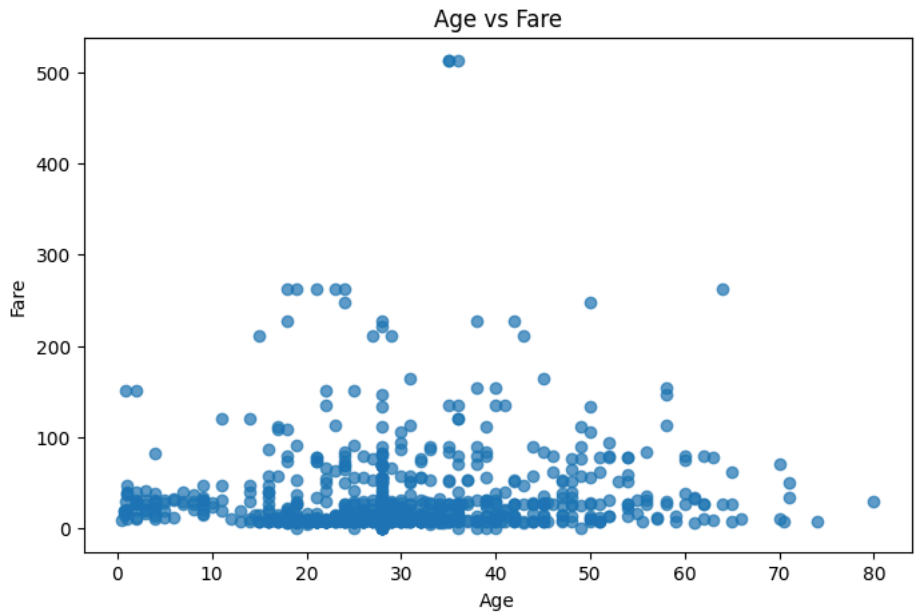
```
plt.figure(figsize=(8, 5))
counts = df_clean["Embarked"].value_counts()
plt.bar(counts.index.astype(str), counts.values)
plt.title("Passenger Count by Embarkation Port")
plt.xlabel("Embarked")
plt.ylabel("Count")
plt.show()
```



Observation: Southampton (S) is by far the most common embarkation port.

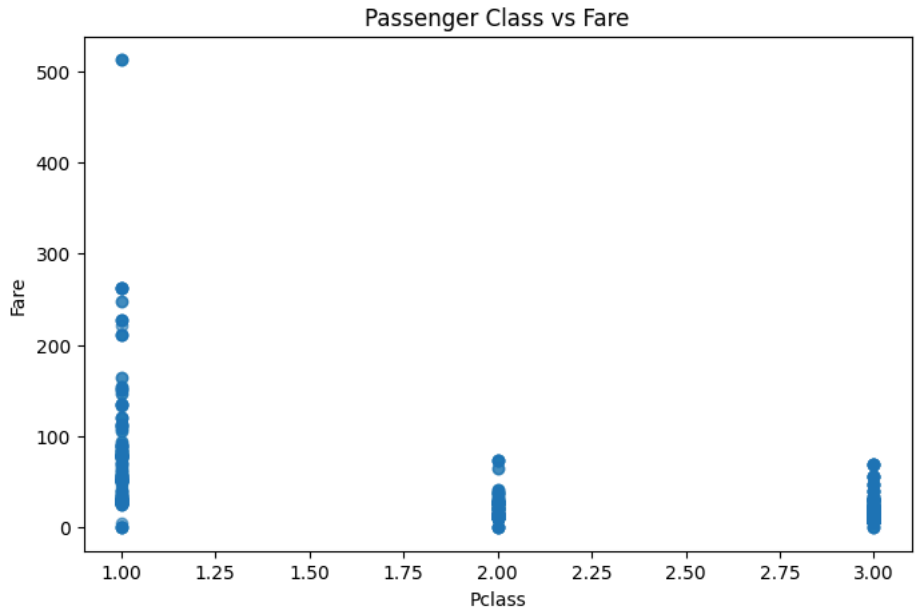
Task 21 – Bivariate Analysis: Numerical vs Numerical

```
plt.figure(figsize=(8, 5))
plt.scatter(df_clean["Age"], df_clean["Fare"], alpha=0.7)
plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.show()
```



Relationship: Age and Fare show no strong linear relationship; fare varies widely across similar ages.

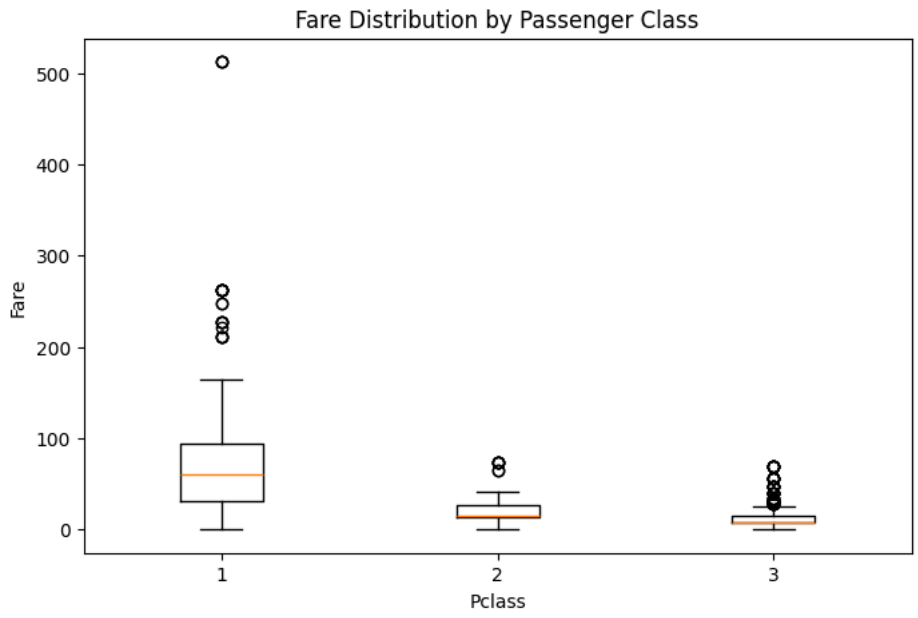
```
plt.figure(figsize=(8, 5))
plt.scatter(df_clean["Pclass"], df_clean["Fare"], alpha=0.6)
plt.title("Passenger Class vs Fare")
plt.xlabel("Pclass")
plt.ylabel("Fare")
plt.show()
```



Relationship: lower class number (especially first class) generally corresponds to higher fares, so the relationship is clearly class-dependent rather than a simple continuous trend.

Task 22 – Categorical vs Numerical

```
plt.figure(figsize=(8, 5))
groups = [df_clean.loc[df_clean["Pclass"] == c, "Fare"] for c in sorted(df_clean["Pclass"].unique())]
plt.boxplot(groups, tick_labels=[str(c) for c in sorted(df_clean["Pclass"].unique())])
plt.title("Fare Distribution by Passenger Class")
plt.xlabel("Pclass")
plt.ylabel("Fare")
plt.show()
```



First-class passengers generally paid much higher fares and show a wider fare range. Third-class fares are concentrated at much lower values.

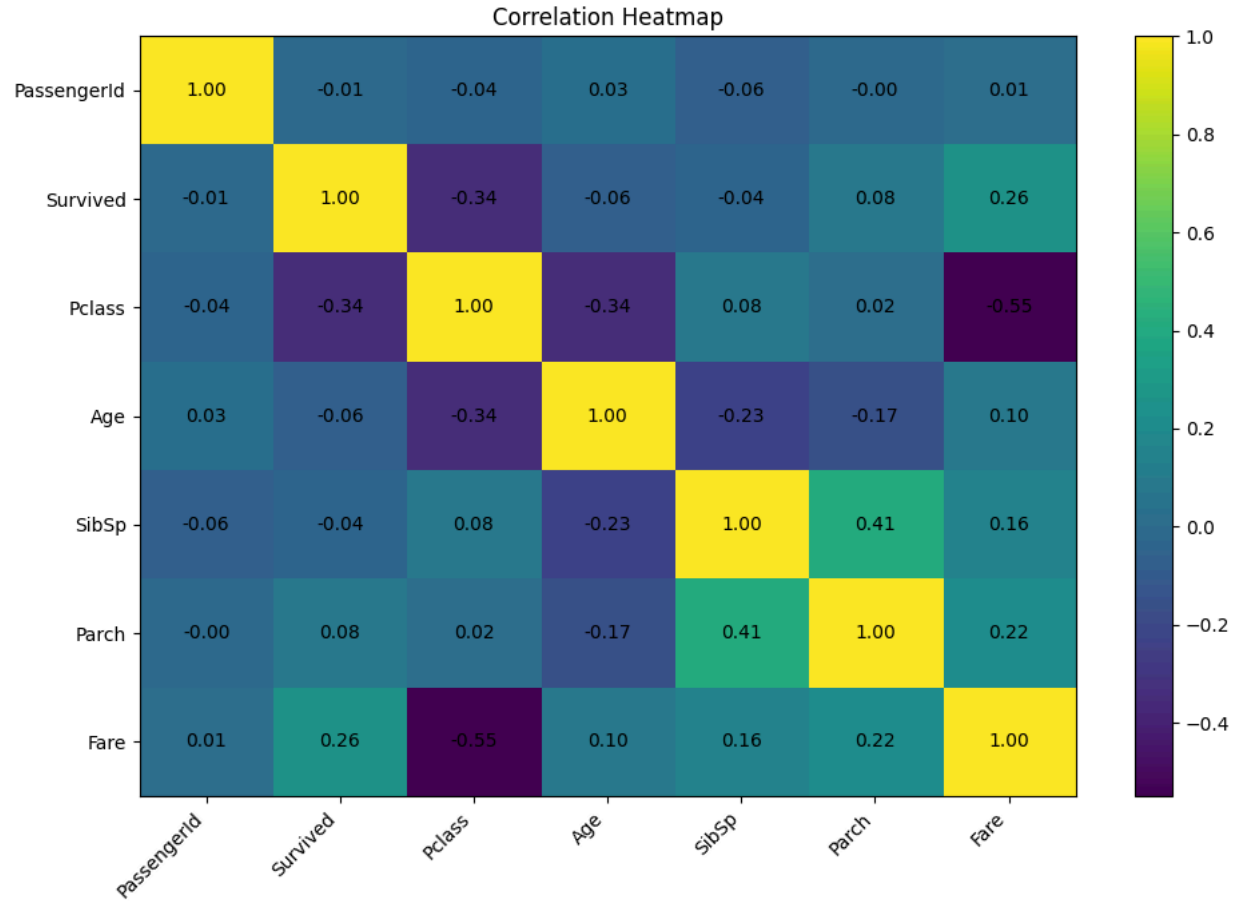
Task 23 – Correlation Analysis

```
corr = df_clean.corr(numeric_only=True)
corr
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.034212	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.064910	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.339898	0.083081	0.018443	-0.549500
Age	0.034212	-0.064910	-0.339898	1.000000	-0.233296	-0.172482	0.096688
SibSp	-0.057527	-0.035322	0.083081	-0.233296	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.172482	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096688	0.159651	0.216225	1.000000

Next steps: Generate code with corr | New interactive sheet

```
plt.figure(figsize=(10, 7))
img = plt.imshow(corr.values, aspect="auto")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr.index)), corr.index)
for i in range(len(corr.index)):
    for j in range(len(corr.columns)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center")
plt.title("Correlation Heatmap")
plt.colorbar(img)
plt.tight_layout()
plt.show()
```




```
target_corr = corr["Survived"].drop("Survived").sort_values()
print("Correlations with Survived:")
print(target_corr)

print("\nStrongest positive correlation with Survived:",
      target_corr.idxmax(), round(target_corr.max(), 3))
print("Strongest negative correlation with Survived:",
      target_corr.idxmin(), round(target_corr.min(), 3))
```

```
Correlations with Survived:
Pclass        -0.338481
Age           -0.064910
SibSp         -0.035322
PassengerId   -0.005007
Parch          0.081629
```

```
Fare           0.257307
Name: Survived, dtype: float64

Strongest positive correlation with Survived: Fare 0.257
Strongest negative correlation with Survived: Pclass -0.338
```

Correlation interpretation

Among raw numerical features, `Fare` has the strongest positive correlation with survival, while `Pclass` has the strongest negative correlation with survival. `PassengerId` and `SibSp` have correlations close to zero with the target, indicating little linear relationship.

Task 24 – Data Type Conversion and Preprocessing Plan

```
preprocessing_plan = pd.DataFrame([
    ["PassengerId", "Identifier", "Drop from ML feature set; no predictive meaning intended"],
    ["Survived", "Target", "Keep as binary target"],
    ["Pclass", "Categorical/Ordinal", "Keep numeric because class order is meaningful"],
    ["Name", "Identifier/Text", "Drop from basic ML-ready table; very high cardinality"],
    ["Sex", "Categorical", "Label encode because it is binary"],
    ["Age", "Numerical", "Median imputation + StandardScaler"],
    ["SibSp", "Numerical discrete", "Keep"],
    ["Parch", "Numerical discrete", "Keep"],
    ["Ticket", "Identifier/Text", "Drop from basic ML-ready table; high cardinality"],
    ["Fare", "Numerical", "Keep + StandardScaler"],
    ["Cabin", "Categorical/Text", "Use derived Deck; then drop original high-cardinality field"],
    ["Embarked", "Categorical nominal", "Mode imputation + One-Hot Encoding"],
    ["Deck", "Categorical nominal", "One-Hot Encoding"]
], columns=["Column", "Type", "Preprocessing"])
preprocessing_plan
```

	Column	Type	Preprocessing
0	PassengerId	Identifier	Drop from ML feature set; no predictive meanin...
1	Survived	Target	Keep as binary target
2	Pclass	Categorical/Ordinal	Keep numeric because class order is meaningful
3	Name	Identifier/Text	Drop from basic ML-ready table; very high card...
4	Sex	Categorical	Label encode because it is binary
5	Age	Numerical	Median imputation + StandardScaler
6	SibSp	Numerical discrete	Keep
7	Parch	Numerical discrete	Keep
8	Ticket	Identifier/Text	Drop from basic ML-ready table; high cardinality
9	Fare	Numerical	Keep + StandardScaler
10	Cabin	Categorical/Text	Use derived Deck; then drop original high-card...
11	Embarked	Categorical nominal	Mode imputation + One-Hot Encoding
12	Deck	Categorical nominal	One-Hot Encoding

Next steps: Generate code with `preprocessing_plan` New interactive sheet

Task 25 – Categorical Encoding

```
df_model = df_clean.copy()

# Label Encoding is appropriate for binary Sex.
le = LabelEncoder()
df_model["Sex"] = le.fit_transform(df_model["Sex"])
print("Sex classes used by LabelEncoder:", list(le.classes_))

# Remove identifiers/high-cardinality raw text after their information has been assessed.
df_model.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"], inplace=True)

# One-Hot Encoding for nominal categories.
df_model = pd.get_dummies(
    df_model,
    columns=["Embarked", "Deck"],
    drop_first=True,
    dtype=int
)
```

```
df_model.head()
```

```
Sex classes used by LabelEncoder: ['female', 'male']
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S	Deck_B	Deck_C	Deck_D	Deck_E	Deck_F	Deck_G
0	0	3	1	22.0	1	0	7.2500	0	1	0	0	0	0	0	0
1	1	1	0	38.0	1	0	71.2833	0	0	0	1	0	0	0	0
2	1	3	0	26.0	0	0	7.9250	0	1	0	0	0	0	0	0
3	1	1	0	35.0	1	0	53.1000	0	1	0	1	0	0	0	0
4	0	3	1	35.0	0	0	8.0500	0	1	0	0	0	0	0	0

Next steps: Generate code with `df_model` New interactive sheet

Label Encoding vs One-Hot Encoding

- **Label Encoding** replaces categories with integer codes. It is suitable for binary categories such as `Sex`, or for truly ordered categories when the numeric order is meaningful.
- **One-Hot Encoding** creates separate 0/1 indicator columns for nominal categories. It avoids creating a false numeric ranking between categories such as embarkation ports or decks.

Task 26 – Feature Scaling

```
# Standardisation on continuous numerical features used in the final model-ready dataset.
standard_scaler = StandardScaler()
df_model[["Age", "Fare"]] = standard_scaler.fit_transform(df_model[["Age", "Fare"]])

print("Standardised Age/Fare preview:")
display(df_model[["Age", "Fare"]].head())

# Min-Max normalisation demonstration on the cleaned unscaled data.
minmax_scaler = MinMaxScaler()
minmax_demo = pd.DataFrame(
    minmax_scaler.fit_transform(df_clean[["Age", "Fare"]]),
    columns=["Age_MinMax", "Fare_MinMax"]
)

print("Min-Max normalised preview:")
display(minmax_demo.head())
```

```
Standardised Age/Fare preview:
```

	Age	Fare
0	-0.565736	-0.502445
1	0.663861	0.786845
2	-0.258337	-0.488854
3	0.433312	0.420730
4	0.433312	-0.486337

```
Min-Max normalised preview:
```

	Age_MinMax	Fare_MinMax
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

Standardisation vs Normalisation

- **StandardScaler** transforms a feature to approximately mean 0 and standard deviation 1. It is useful for many distance-, gradient-, and regularisation-based models.
- **MinMaxScaler** maps values to a fixed range, usually 0 to 1.
- The final dataset uses StandardScaler for `Age` and `Fare`; Min-Max scaling is demonstrated separately so the original analytical interpretation is not mixed with two different scaling schemes.

Task 27 – Final Dataset Inspection

```
display(df_model.head())
print("Final shape:", df_model.shape)
print("\nFinal info:")
df_model.info()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S	Deck_B	Deck_C	Deck_D	Deck_E	Deck_F	D
0	0	3	1	-0.565736	1	0	-0.502445	0	1	0	0	0	0	0	
1	1	1	0	0.663861	1	0	0.786845	0	0	0	1	0	0	0	
2	1	3	0	-0.258337	0	0	-0.488854	0	1	0	0	0	0	0	
3	1	1	0	0.433312	1	0	0.420730	0	1	0	1	0	0	0	
4	0	3	1	0.433312	0	0	-0.486337	0	1	0	0	0	0	0	

```
Final shape: (891, 17)

Final info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 17 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Survived      891 non-null    int64
 1   Pclass        891 non-null    int64
 2   Sex           891 non-null    int64
 3   Age           891 non-null    float64
 4   SibSp         891 non-null    int64
 5   Parch         891 non-null    int64
 6   Fare          891 non-null    float64
 7   Embarked_Q    891 non-null    int64
 8   Embarked_S    891 non-null    int64
 9   Deck_B        891 non-null    int64
 10  Deck_C        891 non-null    int64
 11  Deck_D        891 non-null    int64
 12  Deck_E        891 non-null    int64
 13  Deck_F        891 non-null    int64
 14  Deck_G        891 non-null    int64
 15  Deck_T        891 non-null    int64
 16  Deck_Unknown  891 non-null    int64
dtypes: float64(2), int64(15)
memory usage: 118.5 KB
```

```
df_model.describe()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S	Deck
count	891.000000	891.000000	891.000000	8.910000e+02	891.000000	891.000000	8.910000e+02	891.000000	891.000000	891.0000
mean	0.383838	2.308642	0.647587	2.272780e-16	0.523008	0.381594	3.987333e-18	0.086420	0.725028	0.0527
std	0.486592	0.836071	0.477990	1.000562e+00	1.102743	0.806057	1.000562e+00	0.281141	0.446751	0.2236
min	0.000000	1.000000	0.000000	-2.224156e+00	0.000000	0.000000	-6.484217e-01	0.000000	0.000000	0.0000
25%	0.000000	2.000000	0.000000	-5.657365e-01	0.000000	0.000000	-4.891482e-01	0.000000	0.000000	0.0000
50%	0.000000	3.000000	1.000000	-1.046374e-01	0.000000	0.000000	-3.573909e-01	0.000000	1.000000	0.0000
75%	1.000000	3.000000	1.000000	4.333115e-01	1.000000	0.000000	-2.424635e-02	0.000000	1.000000	0.0000
max	1.000000	3.000000	1.000000	3.891554e+00	8.000000	6.000000	9.667167e+00	1.000000	1.000000	1.0000

```
print("Missing values in final dataset:", int(df_model.isnull().sum().sum()))
print("Duplicate records in final dataset:", int(df_model.duplicated().sum()))
```

```
Missing values in final dataset: 0
Duplicate records in final dataset: 110
```

Task 28 – Before vs After Comparison

```
before_num = df.select_dtypes(include=np.number).shape[1]
before_cat = df.select_dtypes(exclude=np.number).shape[1]
after_num = df_model.select_dtypes(include=np.number).shape[1]
```

```
after_cat = df_model.select_dtypes(exclude=np.number).shape[1]

comparison = pd.DataFrame({
    "Parameter": [
        "Rows", "Columns", "Missing Values", "Duplicate Records",
        "Numerical Columns", "Categorical Columns"
    ],
    "Before Preprocessing": [
        df.shape[0], df.shape[1], int(df.isnull().sum().sum()),
        int(df.duplicated().sum()), before_num, before_cat
    ],
    "After Preprocessing": [
        df_model.shape[0], df_model.shape[1], int(df_model.isnull().sum().sum()),
        int(df_model.duplicated().sum()), after_num, after_cat
    ]
})
comparison
```

	Parameter	Before Preprocessing	After Preprocessing
0	Rows	891	891
1	Columns	12	17
2	Missing Values	866	0
3	Duplicate Records	0	110
4	Numerical Columns	7	17
5	Categorical Columns	5	0

Next steps: Generate code with `comparison` New interactive sheet

Task 29 – Save the Clean Dataset

```
df_model.to_csv("cleaned_dataset.csv", index=False)

if pd.read_csv("cleaned_dataset.csv").shape == df_model.shape:
    print("cleaned_dataset.csv generated and verified successfully.")
else:
    print("Verification failed.")
```

```
cleaned_dataset.csv generated and verified successfully.
```

Task 30 – EDA Findings

1. The raw dataset contains **891 passengers and 12 attributes**.
2. The overall survival rate is approximately **38.4%**, so non-survivors are the majority.
3. `Age` contains **177 missing values**, `Cabin` contains **687**, and `Embarked` contains **2**.
4. The median passenger age is **28 years**; median imputation is used for missing ages.
5. Southampton (`S`) is the most frequent embarkation port and is used to fill the two missing `Embarked` values.
6. The raw data contains **no exact duplicate rows**.
7. Male passengers are more numerous than female passengers.
8. Third-class passengers form the largest ticket-class group.
9. Fare is strongly right-skewed, with a small number of very expensive tickets.
10. IQR flags outliers in both `Age` and `Fare`, but the detected values are plausible and are retained.
11. Female passengers had a much higher survival rate (about **74.2%**) than male passengers (about **18.9%**).
12. Survival also varies strongly by ticket class: first class had the highest survival rate (about **63.0%**) and third class the lowest (about **24.2%**).
13. Among raw numerical variables, `Fare` has the strongest positive correlation with `Survived`, while `Pclass` has the strongest negative correlation.